

# B2B I

: v1.0 | : 2026-07-27 | : &

## 1.

B2B . 1 , . . .  
(Incremental Cash Margin) .  
 , .

## 2. (Trade-off Matrix)

### 2.1 (Frontend)

| &                                |                         |       |         | (Trade-off)                       |     |           |     |
|----------------------------------|-------------------------|-------|---------|-----------------------------------|-----|-----------|-----|
| React 19 + Vite + TypeScript ( ) | B2B ,                   | SPA,  | HMR     | vs Next.js / API . Vite           | SEO | SSR       | BFF |
| TanStack Query v5 ( )            | API ,                   |       | ,       | vs fetch / .                      |     | API       |     |
| URL Search Params                | (2F~1F), URL            |       | , D-Day | vs In-memory state , URL 100%     |     | URL       |     |
| Zustand ( )                      | UI .                    |       |         | vs React state / selector         |     | UI        |     |
| TanStack Table v8                |                         | SKU , | , ,     | vs Basic Table / AG Grid Headless |     | shadcn/ui |     |
| Recharts                         | vs                      | vs    | ,       | vs Chart.js / D3.js React SVG     |     | D3        |     |
| Tailwind CSS v4 + shadcn/ui ( )  |                         |       | .       | vs MUI /                          |     | .         |     |
| React Hook Form + Zod            |                         |       |         | vs Formik / Plain State od        |     | Zod Z     |     |
| Web Vitals                       | LCP < 1.2s, INP < 100ms |       | UX      | vs                                |     |           |     |
| : Vitest + RTL + Playwright ( )  | .                       | .     | E2E     | vs Jest/Cypress / Vite-React CI   |     | .         |     |

2.2 (Backend)

| &                                      |                             |                                               |                   | (Trade-off) |
|----------------------------------------|-----------------------------|-----------------------------------------------|-------------------|-------------|
| Java 21 + Spring Boot 3.x ( )          | REST API, . . .             | vs Node.js/Python / FastAPI                   | OpenAPI-timeout . | Trace ID    |
| Spring Data JPA + QueryDSL 5.x ( )     | CRUD .                      | vs Native SQL/jOOQ / QueryDSL, read model-SQL | JPA-SQL           | r           |
| MyBatis 3.5+                           | / Function/CTE , Window SQL | vs JPA SQL XML                                | XML               |             |
| (MVP)                                  | → → → → .                   | vs State Machine / num .                      | E . machine       | State M     |
| Spring Security OAuth2/OIDC + RBAC ( ) | .                           | vs JWT / IdP R Resource Server                | IdP .             |             |
| Spring Batch 5 + 1                     | . .                         | vs @Scheduled/Quartz/Kubernetes CronJob / .   | Batch Quartz      |             |
| JUnit 5 + Mockito + Testcontainers     | Docker PostgreSQL/Redis     | vs H2 DB H2 PostgreSQL JSONB/SQL              | Docker            |             |

2.3 (Database & Cache)

| &                  |                       |                         |                   | (Trade-off) |
|--------------------|-----------------------|-------------------------|-------------------|-------------|
| PostgreSQL 16+ ( ) | . . . , payload JSONB | vs MySQL/Oracle / JSONB | . . . .           |             |
| Redis 7.x ( )      | .LLM TTL              | vs / TTL                | . . 0 . .stampede |             |

2.4 AI (AI Service)

| &                                |       |                           |                               | (Trade-off) |
|----------------------------------|-------|---------------------------|-------------------------------|-------------|
| Python 3.11+ + FastAPI ( )       | .ML   | vs Spring AI / Python .ML | HTTP .timeout .CPU            |             |
| OR-Tools/HiGHS PuLP + solver ( ) | . . . | vs / . . MILP             | ‘100% ’ . . infeasible solver |             |
| scikit-learn / LightGBM +        | . .   | vs RAG/Vector DB /        | . . . . .                     |             |
| LLM Provider Adapter ( )         | . .   | vs / . . .                | PII .JSON Schema .fallback .  |             |

2.5 , , Sentry (Observability & Testing)

| &                      |                                         |                        |         | (Trade-off) |
|------------------------|-----------------------------------------|------------------------|---------|-------------|
| Sentry                 | /                                       | vs                     | / , ,   | ( 5,000 )   |
| Prometheus + Grafana   | CPU, , DB , API R PS/Latency            | vs CloudWatch          |         |             |
| Loki + Promtail        | Logback JSON Grafana                    | vs ELK Stack           | 1/10    |             |
| OpenTelemetry + Jaeger | React->Spring Boot->FastAPI->PostgreSQL | vs TraceID             | TraceID |             |
| k6 ( )                 | . . API SLA P95/P99. .                  | vs JMeter/Locust / SLO | 500     |             |

3. AI & 4

1. ML (Feature Engineering) & : , D-Day, 30/60/90 , , LightGBM .
2. / SKU (Cold-Start) : -> -> 65% .
3. LLM (Structured Output) & Redis 0 : LLM JSON , Redis 0 .
4. (Closed-Loop Feedback System): ( , ) DB AI (MAPE) , ML .

4.

Frontend (React SPA) -> Reverse Proxy (Nginx) -> Core Backend (Spring Boot) -> AI Microservice (Python FastAPI) -> Database (PostgreSQL/Redis) -> Observability (Prometheus/Loki/Sentry) .

## 5. 项目架构与技术方案

- 项目架构：采用前后端分离架构，前端使用 React 19 + Vite + TypeScript，后端使用 Spring Boot 3 + FastAPI。数据库采用 PostgreSQL，缓存使用 Redis。部署环境为 Docker + Kubernetes。
- 前端技术栈：React 19 + Vite + TypeScript。SEO 优化使用 Next.js 框架。API 接口使用 Axios 库。状态管理使用 Redux Toolkit。路由管理使用 React Router。打包工具使用 Vite。
- 后端技术栈：Spring Boot 3 + FastAPI。数据库使用 PostgreSQL。ORM 框架使用 JPA/QueryDSL。MyBatis 用于复杂查询。API 接口使用 OpenAPI 3.0 规范。鉴权使用 OAuth2.0。消息队列使用 RabbitMQ。分布式事务使用 Seata。性能优化使用 Redis 缓存。日志使用 ELK 栈。监控系统使用 Prometheus + Grafana。
- 数据库设计：采用数据库设计范式，使用 JPA/QueryDSL 进行 ORM 操作。数据库设计遵循 CRUD 原则。数据库设计使用 QueryDSL 进行复杂查询。数据库设计使用 read model 进行数据建模。数据库设计使用 SQL 进行数据操作。
- 系统架构：采用微服务架构，使用 Spring Cloud 框架。系统架构使用 MVP 模式。系统架构使用 Enum 枚举。系统架构使用 State Machine 状态机。系统架构使用 Quartz 定时任务。系统架构使用 Quartz, @Scheduled 定时任务。系统架构使用 Redis 缓存。系统架构使用 TTL 过期时间。系统架构使用 0 初始值。
- 性能优化：使用 OR-Tools/HiGHS 求解器。性能优化使用 PuLP 求解器。性能优化使用 infeasible-solver fallback。性能优化使用 LightGBM 模型。性能优化使用 '100%' 准确率。性能优化使用 LLM 模型。性能优化使用 Provider Adapter JSON Schema PII 数据。性能优化使用 100% 准确率。

### 5.1 前端技术方案

- 前端框架：使用 React 19 + Vite + TypeScript。前端框架使用 SKU 系统。前端框架使用 IdP/OIDC, RBAC, 鉴权。前端框架使用 fallback, 鉴权。前端框架使用 LLM 模型。前端框架使用 DB 数据库。前端框架使用 timeout, 鉴权。

### 5.2 后端技术方案

- P0: React/Vite 前端框架, Spring API 后端框架, PostgreSQL 数据库, Redis 缓存, Docker 部署, Kubernetes 容器化。
- P1: FastAPI 后端框架, OpenAPI 3.0 规范, OAuth2.0 鉴权, RabbitMQ 消息队列, Seata 分布式事务, Prometheus + Grafana 监控系统。
- P2: LLM 模型, State Machine 状态机, Quartz 定时任务, Quartz, @Scheduled 定时任务, Redis 缓存, TTL 过期时间, 0 初始值。